Assignment 2
Q2 Report

Abhay Sharma 2012CS50272 | Akash Dabaniya 2021CS10111 | Mridul Singh 2018CS50412

March 2026

Disclaimer: This document was typeset in LaTeX with help from an AI language model only for formatting, wording suggestions, and layout.

Task 1: Optimization Formulation

The route blocking problem requires selecting exactly k directed routes to remove from the forest graph so that the expected number of burned regions is minimised. Fire spreads according to the classic **Independent Cascade (IC) model** [1] with independent ignition probabilities on each route.

We first define the key sets and quantities:

- $V = \{1, 2, \ldots, n\}$ set of all forest regions
- $E \subseteq V \times V$ set of all possible directed fire-spread routes
- $p : E \to (0, 1]$ independent ignition probability on each route
- $G = (V, E, p)$ complete probabilistic directed graph
- $A_0 \subseteq V$ given set of initially burning regions
- $R \subseteq E$ any subset of routes we choose to block
- $G_R = (V, E \setminus R, p)$ graph after removing the blocked routes R
- A_∞ random set of regions burning at termination
- $\sigma(R) = \mathbb{E}[|A_\infty|]$ expected number of burned regions

The combinatorial optimisation formulation consists of the following three clearly defined components:

Decision Variables
For each route $e \in E$, let

$$x_e \in \{0, 1\},$$

where $x_e = 1$ if and only if route e is blocked.

Objective Function
Minimise the expected number of burned regions:

$$\min \quad \sigma(R(x)) = \mathbb{E}[|A_\infty|],$$

where $R(x) = \{e \in E \mid x_e = 1\}$.

Constraints
Exactly k routes must be blocked:

$$\sum_{e \in E} x_e = k, \qquad x_e \in \{0, 1\} \quad \forall e \in E.$$

Task 2: NP-Hardness

To prove that the route blocking problem is NP-hard, we give a polynomial-time reduction from the **Minimum k-Union** problem (also known as the Minimum Coverage problem), which is a well-known NP-hard optimisation problem [8, 9].

We first establish NP-hardness for the deterministic special case (where all propagation probabilities are set to $p \equiv 1$), and then use a downward-reduction argument to extend the result to the general stochastic case where $p(e) \in (0, 1]$.

We proceed in two steps:

- First, we prove NP-hardness for the **deterministic special case** where all propagation probabilities are set to $p \equiv 1$.
- Then, we use a downward-reduction argument to show that this implies hardness for the general stochastic case where $p(e) \in (0, 1]$.

This style of reduction is the standard approach used in the foundational influence-minimisation literature [1, 4].

Source Problem: Minimum k-Union
Given a universe $U = \{u_1, \ldots, u_m\}$, a collection of subsets $\mathcal{S} = \{S_1, \ldots, S_n\} \subseteq 2^U$, and an integer k, find a subcollection of exactly k sets whose union has the smallest possible size.

We construct a route-blocking instance in $O(m + n)$ time as follows:

Reduction Construction

- Create one seed node s (so $A_0 = \{s\}$).
- For each subset $S_j \in \mathcal{S}$, create a set-node v_j.
- For each element $u_i \in U$, create an element-node w_i.
- Add a directed edge $s \to v_j$ with probability $p = 1$ for every j.
- For every $u_i \in S_j$, add a directed edge $v_j \to w_i$ with probability $p = 1$.
- Set the blocking budget to exactly k.

In this graph fire starts at s and deterministically activates every v_j node. Each v_j then deterministically activates all w_i nodes belonging to S_j.

Let R be the set of k edges we block (we only block edges of the form $s \to v_j$).

Correctness of the Reduction

- Node s and all v_j nodes always burn.
- An element-node w_i burns if and only if it belongs to at least one *unblocked* set S_j.
- Therefore $\sigma(R) = 1 + n + \left|\bigcup_{S_j \text{ not blocked}} S_j\right|$.

Minimising $\sigma(R)$ is exactly equivalent to choosing k sets to block so that the union of the remaining $(n-k)$ sets is minimised. This is precisely the Minimum k-Union problem.

Since Minimum k-Union is NP-hard and the reduction is polynomial-time, we conclude the following:

- The deterministic version of the route blocking problem (where $p \equiv 1$ on every edge) is NP-hard.
- The deterministic case ($p \equiv 1$ on every edge) is a special case of the general Independent Cascade model where $p(e) \in (0, 1]$.
- Therefore, any polynomial-time algorithm that solves the general stochastic route blocking problem would also solve the deterministic special case in polynomial time.
- Hence, by downward reduction, the general probabilistic route blocking problem is also NP-hard.

Task 3: Structural Properties

Let $\sigma(R)$ denote the expected number of burned regions after blocking the set of routes R. We analyse whether $\sigma(R)$ satisfies the two key properties.

1. Monotone (Yes)

$\sigma(R)$ is monotone non-increasing. For any set of blocked routes $R \subseteq E$ and any route $e \notin R$:

$$\sigma(R \cup \{e\}) \leq \sigma(R)$$

Proof :

- **Probability Space:** Let Ω be the sample space of all possible edge realisations. Each $\omega \in \Omega$ represents a deterministic graph state where every edge $e \in E$ is either "live" (sampled with probability p_e) or "blocked" (with probability $1-p_e$). Let $E_{live}(\omega)$ denote the set of live edges in realisation ω.
- **Deterministic Reachability:** For a fixed set of blocked routes R and a fixed realisation ω, the fire spreads deterministically along the valid edges $E(R, \omega) = E_{live}(\omega) \setminus R$. Let $A_\infty(R, \omega)$ denote the exact set of nodes reachable from A_0 using only the directed edges in $E(R, \omega)$.
- **Subset Property:** Consider blocking one additional edge $e \notin R$. The available edges under the *exact same* realisation ω become $E(R \cup \{e\}, \omega) = E_{live}(\omega) \setminus (R \cup \{e\})$.
- **Path Preservation:** By definition of set difference, $E(R \cup \{e\}, \omega) \subseteq E(R, \omega)$. Because every valid edge under $R \cup \{e\}$ is also valid under R, any directed path from A_0 to a node v in $E(R \cup \{e\}, \omega)$ must also exist in $E(R, \omega)$.
- **Node Set Inclusion:** This strictly implies that the set of burned nodes cannot grow: $A_\infty(R \cup \{e\}, \omega) \subseteq A_\infty(R, \omega)$. Consequently, their cardinalities must satisfy $|A_\infty(R \cup \{e\}, \omega)| \leq |A_\infty(R, \omega)|$.
- **Expectation:** Since this cardinality inequality holds for *every* possible realisation $\omega \in \Omega$, it must also hold for the expected value taken over the probability distribution of Ω:

  $$\mathbb{E}_\omega[|A_\infty(R \cup \{e\}, \omega)|] \leq \mathbb{E}_\omega[|A_\infty(R, \omega)|]$$

  Substituting the definition of the expected spread yields:

  $$\sigma(R \cup \{e\}) \leq \sigma(R)$$

This proves that $\sigma(R)$ is monotone non-increasing.

2. Submodular (No)
$\sigma(\cdot)$ is *not* submodular.
Counterexample:
Consider a directed chain graph with 4 nodes representing a highly unfortunate chain of events:

- s: Cannon (Seed node, $A_0 = \{s\}$)
- u: Dry Bush
- v: Propane Tank
- w: Fireworks Tent

Assume there are three directed routes connecting them in a line ($s \to u \to v \to w$), all with ignition probability $p = 1$. Let's focus on two specific routes:
$e_1 = (s, u)$ and $e_2 = (u, v)$.

- **Base Case** ($\emptyset$): The fire spreads all the way to the fireworks. All 4 nodes burn. $\sigma(\emptyset) = 4$.
- **Block** e_1 ($\{e_1\}$): We stop the cannon from igniting the bush. Only s burns. $\sigma(\{e_1\}) = 1$.
- **Block** e_2 ($\{e_2\}$): The bush burns, but we save the propane tank and fireworks. Nodes s and u burn. $\sigma(\{e_2\}) = 2$.
- **Block both** ($\{e_1, e_2\}$): The fire is stopped at s anyway. Only s burns. $\sigma(\{e_1, e_2\}) = 1$.

Let $A = \emptyset$, $B = \{e_2\}$, and the new route we want to block be $e = e_1$. Note that $A \subseteq B$.

- Marginal change when e_1 is added to A:
  $\sigma(A \cup \{e_1\}) - \sigma(A) = 1 - 4 = \mathbf{-3}$.
- Marginal change when e_1 is added to B:
  $\sigma(B \cup \{e_1\}) - \sigma(B) = 1 - 2 = \mathbf{-1}$.

Since $-3 \ngeq -1$, the submodular inequality:

$$\sigma(A \cup \{e\}) - \sigma(A) \;\geq\; \sigma(B \cup \{e\}) - \sigma(B)$$

is cleanly violated. Blocking e_1 saves 3 nodes initially, but only saves 1 node if e_2 is already blocked. Because the marginal reduction diminishes, $\sigma(\cdot)$ is not submodular.

Task 4: Algorithm Design

The algorithm is based on the Reverse Reachable set sampling framework originally introduced by Borgs et al. [5] and significantly refined for near-optimal performance by Tang et al. [4] in the context of influence maximisation. The same sampling technique has

been widely adapted to influence minimisation (edge/node blocking) problems, including under the Independent Cascade model (see e.g. [7, 6]).

In our setting, we sample random backward reachable sets (RR-sets) starting from uniformly chosen nodes; an RR-set consists of all edges traversed in a backward simulation until a seed node from A_0 is reached (respecting the h-hop limit when applicable). Each valid RR-set represents a minimal set of edges whose simultaneous presence allows fire to reach the sampled node from the seeds in one possible realisation. The problem then reduces to finding a set of k edges that collectively intersect (cover) the largest possible number of these RR-sets. This coverage objective is greedily maximised using the standard submodular maximisation greedy procedure.

1. Pseudocode

1. Generate M random RR-sets ($M \approx$ 50,000–100,000):
   - Pick a random node $v \in V$.
   - Perform backward BFS from v, sampling each incoming edge with probability p_e.
   - Stop when a seed node from A_0 is reached (respecting the h-hop limit if given).
   - Record the unique list of traversed edges as one RR-set.
2. For each edge e, count how many RR-sets contain it.
3. Greedily select k edges:
   - Repeatedly pick the edge that covers the most yet-uncovered RR-sets.
   - Mark all RR-sets containing that edge as covered.
   - Add the edge to the blocking set R and output the pair (u, v).
4. If fewer than k edges were selected, fill arbitrarily.

2. Computational Complexity

Let $n = |V|$, $m = |E|$, M = number of simulated RR-sets, k = budget, and W = average number of edges visited per backward BFS. The complexity is derived in the following discrete steps:

- **Step 1: RR-Set Generation.** The algorithm performs M independent reverse Breadth-First Searches. A standard BFS traverses at most all vertices and edges, taking $\mathcal{O}(n + m)$ time in the absolute worst case. However, by stopping early upon reaching a seed node or hitting the h-hop limit, it explores an average of W edges. Thus, the expected time to generate all sets is $\mathcal{O}(M \cdot W)$.
- **Step 2: Coverage Initialization.** Calculating the initial coverage score for all m edges requires iterating through all edges in the M generated RR-sets, which takes $\mathcal{O}(M \cdot W)$ time.
- **Step 3: Greedy Edge Selection.** We must select k routes to block. This selection involves exactly k iterations. In each iteration, scanning the active coverage array to find the edge with the maximum score takes $\mathcal{O}(m)$ time, contributing $\mathcal{O}(k \cdot m)$ across all steps.
- **Step 4: Coverage Updates.** When an edge is selected, the coverage scores of other edges in the newly covered RR-sets must be decremented. Since an RR-set is processed and marked "covered" at most once during the entire algorithm, the total aggregate time for all updates across all k steps is strictly bounded by the total sum of the sizes of the RR-sets, which is $\mathcal{O}(M \cdot W)$.

Total Expected Time Complexity: $\mathcal{O}(M \cdot W + k \cdot m)$.
Total Worst-Case Time Complexity: $\mathcal{O}(M(n + m) + k \cdot m)$.
Total Space Complexity: $\mathcal{O}(n + m + M \cdot W)$ to store the graph's reverse adjacency list and the generated RR-sets.

3. Approximation Guarantee The algorithm provides a formal $\left(1-\frac{1}{e}\right)$ approximation guarantee with respect to the empirical RR-set coverage objective.

Proof :

Let $\mathcal{R}$ be the finite collection of all M generated RR-sets. For any blocking set $R \subseteq E$, define the surrogate objective

$$f_{RR}(R) \;=\; \left|\{\text{RR-set } S \in \mathcal{R} : S \cap R \neq \emptyset\}\right|$$

(i.e., the number of RR-sets that contain at least one edge from R).

Step 1. Monotonicity of f_{RR}. If $R \subseteq R'$, then every RR-set covered by R is also covered by R'. Hence $f_{RR}(R) \leq f_{RR}(R')$.

Step 2. Submodularity of f_{RR}. Let $A \subseteq B \subseteq E$ and $e \notin B$. The marginal gain of adding e to A is the number of RR-sets that contain e but are not yet covered by A. The marginal gain when added to B is the number of RR-sets that contain e but are not yet covered by B. Since B already covers at least as many RR-sets as A, the set of newly covered RR-sets when adding e to B is a subset of those when adding e to A. Therefore

$$f_{RR}(A \cup \{e\}) - f_{RR}(A) \;\geq\; f_{RR}(B \cup \{e\}) - f_{RR}(B).$$

Thus f_{RR} is monotone and submodular.

Step 3. Standard Greedy Theorem. For any monotone submodular function f and cardinality constraint $|R| \leq k$, the greedy algorithm (repeatedly picking the element with maximum marginal gain) returns a set R_G satisfying

$$f(R_G) \geq \left(1 - \frac{1}{e}\right) f(R^*),$$

where R^* is any optimal set of size k. (Classic result of Nemhauser, Wolsey & Fisher, 1978.)

Step 4. Application. Our algorithm exactly implements this greedy procedure on f_{RR}. Therefore

$$f_{RR}(R_G) \geq \left(1 - \frac{1}{e}\right) f_{RR}(R^*).$$

Step 5. Asymptotic correctness. It is known (Tang et al., SIGMOD 2014) that as the number of RR-sets $M \to \infty$, the surrogate $f_{RR}(R)$ converges (in probability) to the true reduction $\sigma(\emptyset) - \sigma(R)$. Consequently, maximising f_{RR} asymptotically minimises the true objective $\sigma(R)$.

Hence the algorithm guarantees at least $\approx 63.2\%$ of the optimal reduction in expected burned regions.

Task 5: Competitive Evaluation

The submitted solution was implemented in C++ and wrapped in the required `forest_fire.sh` script. The program follows the exact experimental protocol and running command specified in the assignment.

How the experiment was run (according to original instructions):

- The program is executed using the exact command format given in the assignment:

```
bash forest_fire.sh <absolute_path_to_graph> <absolute_path_to_seed_set> <absolute_out
```

- For each candidate blocking set R, $\sigma(R)$ is estimated by generating $r = n_random_instances$ independent random realisations of the remaining graph.
- The final number of burned regions $|A_\infty|$ is recorded for each realisation, and $\sigma(R)$ is computed as the average:

$$\sigma(R) = \frac{1}{r} \sum_{i=1}^{r} |A_\infty^{(i)}|$$

- The output file must contain exactly k lines, each with a pair of integers $(u\ v)$ representing a blocked route.
- The official `evaluate.sh` script (provided in the Eval folder) was used to compute both $\sigma(\emptyset)$ (fixed baseline) and $\sigma(R)$ using the same simulation protocol for fair comparison across all submissions.

Results on Public Datasets

Table 1: Performance on the two public datasets using the official evaluation script.

Dataset	k	hops	Reduction ratio $\rho(R)$
Public Dataset 1	50	-1	.688
Public Dataset 2	20	3	.781

References

[1] D. Kempe, J. Kleinberg, and É. Tardos, "Maximizing the spread of influence through a social network," in *Proc. KDD*, 2003.

[2] Y. Tang et al., "Influence maximization: Near-optimal time complexity meets practical efficiency," in *Proc. SIGMOD*, 2014.

[3] T. Hastie et al., *The Elements of Statistical Learning*, Springer, 2009.

[4] Y. Tang, Y. Shi, and X. Xiao, "Influence maximization: Near-optimal time complexity meets practical efficiency," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2014, pp. 75–86.

[5] C. Borgs, M. Brautbar, J. Chayes, and B. Luce, "Maximizing social influence in nearly optimal time," in *Proc. 25th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2014, pp. 946–957.

[6] Y. Tang, X. Xiao, and Y. Shi, "Influence maximization: Near-optimal time complexity meets practical efficiency," ACM Transactions on Knowledge Discovery from Data, vol. 10, no. 2, pp. 1–28, 2015.

[7] H. T. Nguyen, M. T. Thai, and T. N. Dinh, "Stop-and-stare: Optimal sampling algorithms for viral marketing," in *Proc. ACM SIGMOD*, 2016, pp. 1–16.

[8] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, Springer, 1972, pp. 85–103.

[9] D. S. Hochbaum, "Approximation algorithms for the set covering and vertex cover problems," *SIAM Journal on Computing*, vol. 11, no. 3, pp. 555–556, 1982.